

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

DEPTH FIRST SEARCH, TOPOLOGICAL SORT, AND BELLMAN FORD

DEPTH FIRST SEARCH

DEPTH FIRST SEARCH

The traversal skeleton is unchanged. The queue is replaced by the call stack, and the visit order changes completely.

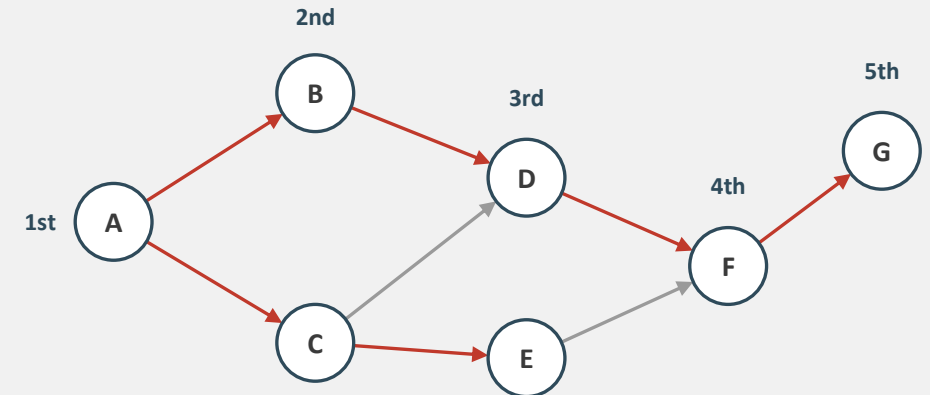
```
def dfs(adj, u, visited):  
    visited.add(u)           # mark on discovery  
    for v in adj[u]:  
        if v not in visited:  
            dfs(adj, v, visited)  
  
def dfs_all(adj):  
    visited = set()  
    for u in adj:  
        if u not in visited:  
            dfs(adj, u, visited)  
    return visited
```

WHAT THE RECURSION IS DOING

One call to `dfs(u)` does not return until every vertex reachable from `u` has been marked. The for loop over `adj[u]` pauses at the first unvisited neighbor and resumes only after that entire branch is exhausted.

`dfs` reaches exactly the vertices reachable from `u`, the same set breadth first search reaches. The red edges on the right are the ones the recursion followed; the faded ones were examined and skipped.

Visit order from A: A, B, D, F, G, C, E



STILL $\Theta(N + M)$

A vertex is marked once, so at most n calls happen. One call to `dfs(u)` scans `adj[u]` once, so the total scanning is the sum of `deg(u)`, which is $2m$ undirected and m directed.

RECURSION DEPTH IS A REAL LIMIT

On a path graph the recursion reaches depth n . CPython raises `RecursionError` past roughly one thousand frames, so a large graph needs the explicit stack version instead.

VERTEX STATE AND TIMESTAMPS

One boolean is enough to finish a traversal. Three states and a clock is what lets depth first search describe the structure it walked through.

```
RED, BLUE, GREEN = 0, 1, 2

def visit(u):
    color[u] = BLUE                # call is now open
    clock[0] += 1; disc[u] = clock[0]

    for v in adj[u]:
        if color[v] == RED:
            visit(v)

    color[u] = GREEN                # call is closing
    clock[0] += 1; finish[u] = clock[0]
```

THE THREE STATES

COLOR	MEANING	WHERE THE VERTEX IS
RED	Not yet discovered	Nowhere
BLUE	Discovered, not yet finished	On the call stack right now
GREEN	Finished	Fully explored, call returned

These three colors are not a graph coloring. Adjacent vertices routinely share one. The color records where the traversal is, not a constraint between neighbors.

VERTEX	DISC	FINISH
A	1	14
B	2	9
C	10	13
D	3	8
E	11	12
F	4	7
G	5	6

BLUE MEANS ON THE CALL STACK

While visit(u) is running, u is blue, and so is every vertex whose call is still open beneath it. Those are exactly the ancestors of u in the recursion.

INTERVALS NEST, NEVER OVERLAP

Read [disc[u], finish[u]] as an interval. Two such intervals are either disjoint or one sits entirely inside the other.

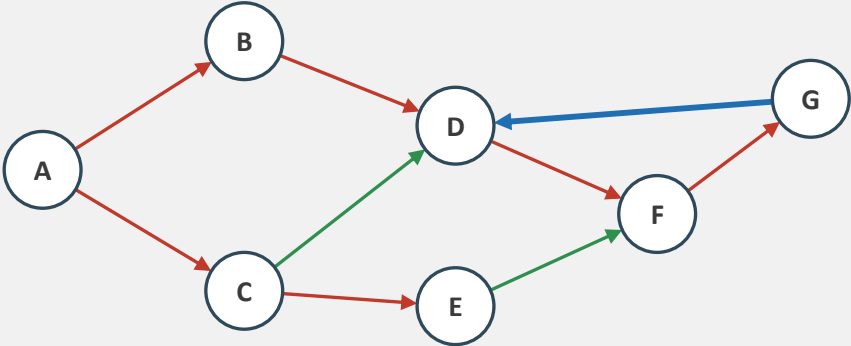
BACK EDGES AND DIRECTED CYCLES

An edge into a blue vertex points back at something still on the call stack, and that is a directed cycle. Nothing else is.

```
def has_cycle(adj):
    # directed graph
    color = {u: RED for u in adj}

    def visit(u):
        color[u] = BLUE
        for v in adj[u]:
            if color[v] == BLUE:
                # back edge found
                return True
            if color[v] == RED and visit(v):
                return True
        color[u] = GREEN
        return False

    return any(visit(u) for u in adj if color[u] == RED)
```



Blue back edge G to D closes D, F, G, D

GREEN IS NOT A CYCLE

Testing v in visited instead of color[v] == BLUE reports a cycle on any diamond. Reaching a GREEN vertex a second time is normal in an acyclic graph.

UNDIRECTED GRAPHS NEED ONE GUARD

Every undirected edge is examined from both ends, so the edge you arrived on always points at a blue vertex. Pass the parent down and skip it.

COLOR OF V	NAME OF EDGE (U, V)	WHAT IT TELLS YOU
RED	Tree edge	v is discovered through this edge
BLUE	Back edge	v is an ancestor of u: a cycle
GREEN	Forward or cross edge	v was already finished: no cycle

PROBLEM: FIND THE CIRCULAR IMPORT

You have a set of source files and, for each one, the modules it imports. Report an actual cycle if one exists, not just the fact that one exists.

01 Vertices are modules, edges are imports.

The relation is not symmetric, so the graph is directed. Nothing is weighted.

02 Keep the blue chain, not just the blue flag.

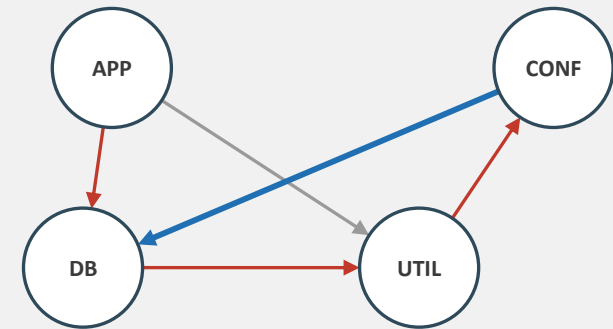
Push u on a list when it turns blue, pop it when it turns green.

03 Slice the chain when a back edge appears.

An edge to a blue v means v is on that list. Slice from v , then append v .

```
def find_cycle(adj):
    color, stack = {u: RED for u in adj}, []

    def visit(u):
        color[u] = BLUE; stack.append(u)
        for v in adj[u]:
            if color[v] == BLUE:
                return stack[stack.index(v):] + [v]
            if color[v] == RED:
                found = visit(v)
                if found: return found
        color[u] = GREEN; stack.pop()
```



Blue back edge closes DB, UTIL, CONF, DB

RUNNING TIME

The traversal is untouched, so it stays $\Theta(n + m)$. The `stack.index` call happens at most once over the whole run, because the function returns immediately after it.

SAME SHAPE, DIFFERENT PROBLEM

Deadlock detection on a wait for graph, reference cycles in a garbage collector, and recursive foreign keys in a schema are all this exact traversal.

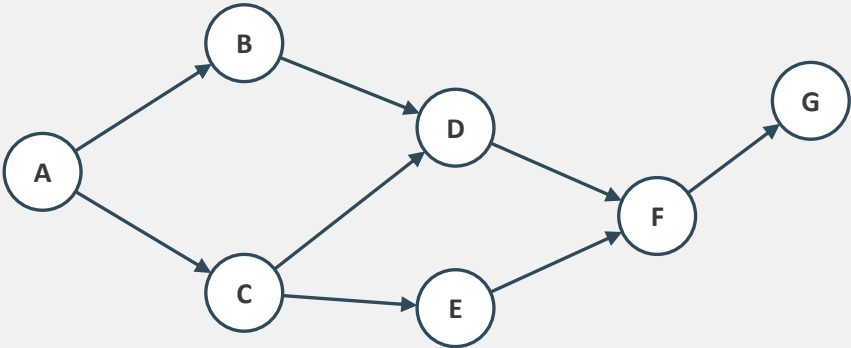
TOPOLOGICAL SORT

DIRECTED ACYCLIC GRAPHS AND LINEAR ORDERS

A topological order lists every vertex so that every edge points forward in the list. It exists exactly when the graph has no directed cycle.

Definition.

An ordering $v_1, v_2, \dots, v_n$ of V is a topological order of the directed graph G if for every edge (v_i, v_j) in E we have $i < j$. A directed graph with no directed cycle is called a directed acyclic graph, or DAG.



STATEMENT

REASON IN ONE LINE

A cycle rules out any order

Each vertex on the cycle would have to precede the next, and the last would have to precede the first.

Every DAG has a vertex of indegree 0

Walk backwards along incoming edges. With no cycle you cannot repeat a vertex, so the walk stops.

Every DAG has a topological order

Take a vertex of indegree 0, put it first, delete it, and the rest is still a DAG.

The order is rarely unique

Vertices with no directed route between them can appear in either relative position.

TWO VALID ORDERS FOR THIS GRAPH

A, B, C, D, E, F, G

A, C, E, B, D, F, G

Check every edge against both. B and C have no route between them, so either can come first.

A IS A SOURCE, G IS A SINK

A source has indegree 0 and must be available to go first. A sink has outdegree 0 and can always go last.

TOPOLOGICAL SORT FROM FINISH TIMES

Run depth first search, append each vertex to a list at the moment it finishes, then reverse the list.

```
def topo_dfs(adj):
    color = {u: RED for u in adj}
    order = []

    def visit(u):
        color[u] = BLUE
        for v in adj[u]:
            if color[v] == RED:
                visit(v)
        color[u] = GREEN
        order.append(u)          # append on FINISH

    for u in adj:
        if color[u] == RED:
            visit(u)

    order.reverse()
    return order
```

STEP	RESULT ON THE RUNNING GRAPH
Finish order	G, F, D, B, E, C, A
Reversed	A, C, E, B, D, F, G
Check	All eight edges point forward

WHY FINISH AND NOT DISCOVERY

When u finishes, every vertex reachable from u has already finished, so all of them are already in the list ahead of u. Reversing puts u first.

Appending on discovery is wrong. On A to B, A to C, C to B, discovery order A, B, C places B before C, but C has an edge into B.

THIS VERSION ASSUMES ACYCLIC

Given a cycle it still returns a list, and that list is simply wrong. Add the BLUE test from the cycle detection code if the input is not known to be a DAG.